



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

**«МИРЭА - Российский технологический университет»**

**РТУ МИРЭА**

---

**Институт информационных технологий (ИИТ)  
Кафедра математического обеспечения и стандартизации  
информационных технологий (МОСИТ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ**  
по дисциплине «Тестирование и верификация программного обеспечения»

**Практическая работа № 3  
ПОДХОДЫ ДЛЯ РАЗРАБОТКИ ПРОГРАММНОГО  
ОБЕСПЕЧЕНИЯ**

Студенты группы *ИКБО-50-23, Астахов С.П.*

\_\_\_\_\_  
(подпись)

Преподаватель *Ильичев Г.П.*

\_\_\_\_\_  
(подпись)

Отчет представлен «\_\_\_»\_\_\_\_\_2025 г.

Москва 2025 г.

**Цель работы:** Изучение и применение различных подходов к разработке программного обеспечения, основанного на тестировании, для повышения качества, надёжности и поддерживаемости кода.

Для достижения поставленной цели работы необходимо выполнить ряд задач:

1. Изучить теоретические основы методологий тестирования: TDD, ATDD, BDD и SDD;
2. Исследовать преимущества и недостатки каждого подхода;
3. Реализовать практические примеры для каждого метода;
4. Проанализировать влияние интеграции тестирования на архитектуру и качество программного продукта;
5. Подготовить итоговый отчёт с выводами и рекомендациями по интеграции подходов в современные процессы разработки.

В данной практической работе был выбран 2 варианта:

## **02. Конвертер единиц измерения**

Функции — преобразование температуры (Цельсий ↔ Фаренгейт), длины, веса.

**TDD** — тесты для корректности конвертации каждой единицы.

**ATDD** — приёмочные тесты для проверки пользовательских сценариев конвертации.

**BDD** — сценарий «Given я ввожу 100 Цельсий, When я выбираю конвертацию в Фаренгейт, Then я получаю 212°F».

**SDD** — спецификация с таблицами входных значений и результатов.

# 1. ПРАКТИЧЕСКАЯ ЧАСТЬ

## 1.1. TDD

### 1.1.1. Создание теста

```
1 import unittest
2 from converter import Converter • "Converter" is unknown import symbol
3
4
5 class TestConverter(unittest.TestCase):
6     def setUp(self):
7         self.obj = Converter()
8
9     def test_convert_length(self):
10        self.assertEqual(self.obj._convert_length(10, "km", "m"), 10_000)
11
12    def test_convert_degrees(self):
13        self.assertEqual(self.obj._convert_degrees(10, "c", "f"), 50)
14
15    def test_convert_weight(self):
16        self.assertEqual(self.obj._convert_weight(10, "kg", "gm"), 10_000)
17
18
19 if __name__ == "__main__":
20     unittest.main()
```

Рисунок 1 — Реализация tdd на python

### 1.1.2. Запуск теста

```
./Тестирование и верификация программного обеспечения/3 Main ? X python test_TDD.py -v
test_convert_degrees (__main__.TestConverter.test_convert_degrees) ... ERROR
test_convert_length (__main__.TestConverter.test_convert_length) ... ERROR
test_convert_weight (__main__.TestConverter.test_convert_weight) ... ERROR
=====
ERROR: test_convert_degrees (__main__.TestConverter.test_convert_degrees)
-----
Traceback (most recent call last):
  File "/home/serey/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/3/test_TDD.py", line 13, in test_convert_degrees
    self.assertEqual(self.obj._convert_degrees(10, "c", "f"), 50)
    ~~~~~^~~~~~
AttributeError: 'Converter' object has no attribute '_convert_degrees'
=====
ERROR: test_convert_length (__main__.TestConverter.test_convert_length)
-----
Traceback (most recent call last):
  File "/home/serey/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/3/test_TDD.py", line 10, in test_convert_length
    self.assertEqual(self.obj._convert_length(10, "km", "m"), 10_000)
    ~~~~~^~~~~~
AttributeError: 'Converter' object has no attribute '_convert_length'
=====
ERROR: test_convert_weight (__main__.TestConverter.test_convert_weight)
-----
Traceback (most recent call last):
  File "/home/serey/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/3/test_TDD.py", line 16, in test_convert_weight
    self.assertEqual(self.obj._convert_weight(10, "kg", "gm"), 10_000)
    ~~~~~^~~~~~
AttributeError: 'Converter' object has no attribute '_convert_weight'
=====
Ran 3 tests in 0.002s
FAILED (errors=3)
```

Рисунок 2 — Запуск тестов

### 1.1.3. Реализация функций

```
1 class Converter:
2     def _convert_length(self, value, from_unit, to_unit):
3         length = {
4             "mm": 0.001,
5             "cm": 0.01,
6             "dm": 0.1,
7             "m": 1.0,
8             "dam": 10.0,
9             "gm": 100.0,
10            "km": 1000.0,
11        }
12
13        return value * length[from_unit] / length[to_unit]
14
15    def _convert_degrees(self, value, from_unit, to_unit):
16        if from_unit == to_unit:
17            return value
18
19        if from_unit == "c":
20            return (value * 9 / 5) + 32
21        elif from_unit == "f":
22            return (value - 32) * 5 / 9
23
24    def _convert_weight(self, value, from_unit, to_unit):
25        weight = {"gm": 0.001, "kg": 1.0, "h": 100.0, "t": 1000.0}
26
27        return value * weight[from_unit] / weight[to_unit]
```

Рисунок 3 — Создание класса и реализация в нем методов

### 1.1.4. Повторный запуск

```
.../Тестирование и верификация программного обеспечения/3 Main ? X python test_TDD.py -v
test_convert_degrees (__main__.TestConverter.test_convert_degrees) ... ok
test_convert_length (__main__.TestConverter.test_convert_length) ... ok
test_convert_weight (__main__.TestConverter.test_convert_weight) ... ok

-----
Ran 3 tests in 0.000s

OK

.../Тестирование и верификация программного обеспечения/3 Main ? > █
```

Рисунок 4 — Повторный запуск тестов

### 1.1.5. Рефакторинг кода

Добавим некоторые проверки для более точной работы кода.

```
1 class Converter:
2     def _convert_length(self, value, from_unit, to_unit):
3         length = {
4             "mm": 0.001,
5             "cm": 0.01,
6             "dm": 0.1,
7             "m": 1.0,
8             "dam": 10.0,
9             "gm": 100.0,
10            "km": 1000.0,
11        }
12        if from_unit not in length or to_unit not in length:
13            return "Единица измерения не найдена"
14        return value * length[from_unit] / length[to_unit]
15
16    def _convert_degrees(self, value, from_unit, to_unit):
17        if from_unit == to_unit:
18            return value
19
20        if from_unit == "c":
21            return (value * 9 / 5) + 32
22        elif from_unit == "f":
23            return (value - 32) * 5 / 9
24        else:
25            return "Единица измерения не найдена"
26
27    def _convert_weight(self, value, from_unit, to_unit):
28        weight = {"gm": 0.001, "kg": 1.0, "h": 100.0, "t": 1000.0}
29        if from_unit not in weight or to_unit not in weight:
30            return "Единица измерения не найдена"
31        return value * weight[from_unit] / weight[to_unit]
```

Рисунок 5 — Рефакторинг кода

## 1.2. ATDD

### 1.2.1. Создание требований и тестов к ним

Составим несколько требований пользователей и опишем их уже в unittest

```

1 import unittest
2 from converter import Converter
3
4
5 class TestConverter(unittest.TestCase):
6     def setUp(self):
7         self.obj = Converter()
8
9     def test_user_weight(self): # Кондитер хочет взвесить ингредиенты в килограммах,
10        # а весы принимают только граммы
11        result = self.obj.convert("weight", 0.5, "kg", "gm") • Cannot access attribute "convert" for class "
12        self.assertEqual(result, 500)
13
14    def test_user_length(self): # Строитель хочет купить 15 м провода, а продают в дм
15    result = self.obj.convert("length", 15, "m", "dm") • Cannot access attribute "convert" for class "Co
16    self.assertEqual(result, 150)
17
18
19 if __name__ == "__main__":
20     unittest.main()

```

Рисунок 6 — Создания тестов по требованиям

### 1.2.2. Запуск тестов

```

~/Тестирование и верификация программного обеспечения/3 Main ? > python test_ATDD.py -v
test_user_length (__main__.TestConverter.test_user_length) ... ERROR
test_user_weight (__main__.TestConverter.test_user_weight) ... ERROR

=====
ERROR: test_user_length (__main__.TestConverter.test_user_length)
-----
Traceback (most recent call last):
  File "/home/serey/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/3/test_ATDD.py", line 15, in test_user_length
    result = self.obj.convert("length", 15, "m", "dm")
    ~~~~~^~~~~~
AttributeError: 'Converter' object has no attribute 'convert'

=====
ERROR: test_user_weight (__main__.TestConverter.test_user_weight)
-----
Traceback (most recent call last):
  File "/home/serey/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/3/test_ATDD.py", line 11, in test_user_weight
    result = self.obj.convert("weight", 0.5, "kg", "gm")
    ~~~~~^~~~~~
AttributeError: 'Converter' object has no attribute 'convert'

-----
Ran 2 tests in 0.001s

FAILED (errors=2)

```

Рисунок 7 — Запуск тестов

### 1.2.3. Дополнение кода

```

1 class Converter:
2     def __init__(self) -> None:
3         self.converters = {
4             "length": self._convert_length,
5             "degrees": self._convert_degrees,
6             "weight": self._convert_weight,
7         }
8
9     def convert(self, category, value, from_unit, to_unit):
10        if category in self.converters:
11            function = self.converters[category]
12            return function(value, from_unit, to_unit)
13        else:
14            raise ValueError("Такой категории не существует.")
15
16        def _convert_length(self, value, from_unit, to_unit):
17            length = {
18                "mm": 0.001,
19                "cm": 0.01,
20                "dm": 0.1,
21                "m": 1.0,
22                "dam": 10.0,
23                "gm": 100.0,
24                "km": 1000.0,
25            }
26            if from_unit not in length or to_unit not in length:
27                return "Единица измерения не найдена"
28            return value * length[from_unit] / length[to_unit]
29
30        def _convert_degrees(self, value, from_unit, to_unit):
31            if from_unit == to_unit:
32                return value
33
34            if from_unit == "c":
35                return (value * 9 / 5) + 32
36            elif from_unit == "f":
37                return (value - 32) * 5 / 9
38            else:
39                return "Единица измерения не найдена"
40
41        def _convert_weight(self, value, from_unit, to_unit):
42            weight = {"gm": 0.001, "kg": 1.0, "h": 100.0, "t": 1000.0}
43            if from_unit not in weight or to_unit not in weight:
44                return "Единица измерения не найдена"
45            return value * weight[from_unit] / weight[to_unit]
46
47
48    def main():
49        conv = Converter()
50        print(f"C: {10} -> F: {conv.convert('degrees', 10, 'c', 'f')}")
51        print(f"km: {7} -> m: {conv.convert('length', 7, 'km', 'm')}")
52
53
54    if __name__ == "__main__":
55        main()

```

Рисунок 8 — Дополнение метода

## 1.2.4. Повторный запуск тестов

```
.../Тестирование и верификация программного обеспечения/3 Main ? > python test_ATDD.py -v
test_user_length (__main__.TestConverter.test_user_length) ... ok
test_user_weight (__main__.TestConverter.test_user_weight) ... ok

-----
Ran 2 tests in 0.000s

OK
```

Рисунок 9 — Запуск тестов

## 1.3. BDD

### 1.3.1. Создание сценариев

Листинг 1 — Сценарии

```
feature: Конвертация градусов

scenarios: Ученому надо преобразовать градусы
    Given я ввожу 100 Цельсий
    When я выбираю конвертацию Фаренгейт
    Then я получаю 212 F
```

### 1.3.2. Создания тестов к сценарию с given, when, then

```
1  import unittest
2  from converter import Converter
3
4
5  class TestConverter(unittest.TestCase):
6      def test_water_boiling_temperature_conversion(self):
7          # Given
8          converter = Converter()
9          input_value = 100
10         from_unit = "c"
11         to_unit = "f"
12
13         # When
14         result = converter.convert("degrees", input_value, from_unit, to_unit)
15
16         # Then
17         self.assertEqual(result, 212)
18
19
20 if __name__ == "__main__":
21     unittest.main()
```

Рисунок 10 — Создание тестов



### 1.3.3. Запуск теста

```
.../Тестирование и верификация программного обеспечения/3 Main ? > python test_BDD.py -v
test_water_boiling_temperature_conversion (__main__.TestConverter.test_water_boiling_temperature_conversion) ... ok
-----
Ran 1 test in 0.000s
OK
```

Рисунок 11 — Запуск тестов

## 1.4. SDD

### 1.4.1. Таблица спецификаций

Таблица 1 — Примеров

| Тип перевода | Кол-во | Иходная единица | Запрашиваемая единица | Ожидаемый результат |
|--------------|--------|-----------------|-----------------------|---------------------|
| Градусы      | 10     | C               | F                     | 50                  |
| Длина        | 100    | km              | dm                    | 1_000_000           |
| Масса        | 1500   | gm              | h                     | 0.015               |

### 1.4.2. Создание

Листинг 2 — Создание спецификаций по таблице

```
{
  «feature»: конвертация единиц
  «scenarios»: [
    {
      «name»: «Перевод градусов»,
      «given»: [
        «10 градусов Цельсия»
      ],
      «when»: «Перевод градусы Цельсия в Фаренгейт»,
      «then»: [
```

```
        «50 градусов Фарингейт»
    ]
},
{
    «name»: «Перевод длины»,
    «given»: [
        «100 километров»
    ],
    «when»: «Перевод километров в дециметры»,
    «then»: [
        «1000000 дециметров»
    ]
},
{
    «name»: «Перевод массы»,
    «given»: [
        «1500 г»
    ],
    «when»: «Перевод грамм в центеры»,
    «then»: [
        «0.015 ц»
    ]
}
]
```

### 1.4.3. Создание тестов по спецификациям

```
1  import unittest
2  from converter import Converter
3
4
5  class TestConverter(unittest.TestCase):
6      def test_convert_degrees(self):
7          convr = Converter()
8          result = convr.convert("degrees", 10, "c", "f")
9          assert result == 50
10
11     def test_convert_length(self):
12         convr = Converter()
13         result = convr.convert("length", 100, "km", "dm")
14         assert result == 1_000_000
15
16     def test_convert_weight(self):
17         convr = Converter()
18         result = convr.convert("weight", 1500, "gm", "h")
19         assert result == 0.015
20
21
22 if __name__ == "__main__":
23     unittest.main()
```

Рисунок 12 — Создание тестов

### 1.4.4. Запусков тестов

```
.../Тестирование и верификация программного обеспечения/3 Main ? X python test_SDD.py -v
test_convert_degrees (__main__.TestConverter.test_convert_degrees) ... ok
test_convert_length (__main__.TestConverter.test_convert_length) ... ok
test_convert_weight (__main__.TestConverter.test_convert_weight) ... ok

-----
Ran 3 tests in 0.000s

OK
```

Рисунок 13 — Запуск тестов

## **2. АНАЛИЗ И ВЫВОДЫ**

В ходе выполнения практической работы были успешно проведены различные виды тестов, такие как TDD, ATDD, BDD, SDD, исследовали преимущества и недостатки каждого подхода, а также проанализировали интеграцию тестирования на архитектуру.